Product Requirement Document: RENTeasy (B2C)

Owner: Product (reverse-engineered from implemented codebase)
Stakeholders: Founders, Product, Engineering, Design, Ops
Version Date: April 25, 2026

Objective
Provide a trusted P2P rental marketplace where users can:
1. Discover nearby high-value or utility items quickly.
2. Request listed items or post open demand when listings are missing.
3. Coordinate safely through in-app chat and lifecycle tracking.
4. Build trust through profile quality, KYC visibility, and transparent listing details.

Problem Statement
Users who need short-term access to items face:
1. Fragmented local discovery and weak filtering.
2. Trust concerns with unknown counterparties.
3. High coordination overhead from discovery to return.
4. No unified experience for both listing-led and demand-led rental intent.

Assumptions
1. V1 can run on mock/demo authentication.
2. KYC can remain mock while trust UX is still shown.
3. In-app chat + clear lifecycle status improves completion rate.
4. City/locality constraints are adequate for early relevance.

User Type
1. Renter: searches listings, sends listing requests, posts open demand.
2. Lender: publishes listings, responds to incoming demand, manages bookings.
3. Hybrid user: both renter and lender.
4. Admin: moderation/KYC control via backend APIs (frontend admin panel not exposed yet).

Implementation Scope
Implemented
1. Home, explore, category pages, search with filters, listing detail, create listing.
2. Listing request flow and open item-request flow.
3. Conversation layer with system and user messages.
4. Renter and lender dashboards with status progression actions.
5. Profile/account update, password update, mock KYC flow.
6. Public requests feed, matching listings, response-to-chat handoff.
7. Trust/safety components and support/legal informational pages.

Partially Implemented
1. Admin backend routes are implemented.
2. Admin frontend route redirects to home.

Not Implemented
1. Payments, refunds, ledger, reconciliation engine.
2. Notification infra (push/SMS/email).
3. AI assistant, reels/video commerce, rich CMS module.

System Specifications
1) What the user sees today
1. Navbar with city selector, geolocation detection, search input and suggestions.
2. Home hero, category showcase, and city-aware fresh listings feed.
3. Search page with filter-sidebar layout (desktop + mobile drawer).
4. Floating quick-action button with:
   1. List Item

   2. Request Item
5. Chat, requested-items, public-requests, renter/lender dashboards.
6. Help, Privacy, Terms, Contact pages.

2) System architecture
`Next.js App (App Router + React Query + motion)`
`-> API client layer (fetch wrapper)`
`-> Express API server (auth, listings, requests, item-requests, conversations, users, kyc, admin, lender, uploads)`
`-> MongoDB (Mongoose models + indexes)`

3) Backend services/modules
1. Auth service (mock login and bearer token auth middleware).
2. Listing service (CRUD + facets + search query parser).
3. Rental request service (listing-specific lifecycle).
4. Open item-request service (demand posting + lender response).
5. Conversation service (chat and system message sequencing).
6. User/profile service (profile updates, password updates).
7. KYC mock service.
8. Lender opportunity feed (open demand scoped by lender inventory footprint).
9. Upload service (image upload to local filesystem path served by backend).

Feature Specification
1) Virtual Assistant
Current
Not implemented as LLM assistant.

Existing substitute capability
1. Search suggestions in navbar.
2. Faceted search and filter refinement.
3. Quick reply chips in chat.

Recommended V2 flow
`User query -> Intent parser -> Listings/Categories/Facets APIs -> Ranked response -> Search or Request CTA`

2) Video Commerce
Current
Not implemented.

Recommended V2 flow
`Reel feed -> Tagged listing/request -> CTA -> Listing detail or Request wizard -> Chat`

3) Booking Confirmation Process (implemented)
Listing request flow
`Listing detail -> Request modal submit -> RentalRequest(status=requested) + Conversation created -> Chat -> Accept/Reject -> Confirm -> Active -> Return pending -> Completed/Disputed/Cancelled`

Open request flow
`Request Item wizard -> ItemRequest(status=open) -> Lender responds with own listing -> Conversation -> Accepted/Confirmed/Active/Completed`

State transition enforcement
1. Enforced by backend status machine for `RentalRequest`.
2. Enforced by backend status machine for `ItemRequest`.

4) Payment Process
Current
No integrated payment rail.

Available payment-adjacent data
1. Rent amount (`quotedRent`/`budgetAmount`).
2. Deposit amount or deposit preference.
3. Status and timestamp trail.

Recommended V2 flow
`Accept request -> Payment authorization -> Booking confirmation -> Active -> Co
mpletion -> Settlement/refund`

5) Reconciliation Flow
Current
No reconciliation engine implemented.

Recommended architecture
`Booking events + payment gateway reports + adjustments -> normalize -> match ->
 exceptions queue -> payout ledger -> audit logs`

6) Revenue Streams
Current
No explicit monetization coded.

Recommended
1. Commission on completed rentals.
2. Delivery facilitation fee.
3. Featured/sponsored listings.
4. Premium trust packages (verification/coverage).

Data Collection
Collected today
1. User profile metadata and trust markers.
2. Listing metadata with pricing options and specifications.
3. Request lifecycles and timestamps.
4. Chat conversation message history.

Missing instrumentation
1. Dedicated analytics event pipeline.
2. Funnel telemetry for conversion stages.

Suggested events
1. `search_submitted`
2. `filter_applied`
3. `listing_viewed`
4. `request_created`
5. `open_request_created`
6. `open_request_responded`
7. `chat_opened`
8. `status_changed`

Database Structure
Collections
1. `users`
2. `listings`
3. `rentalrequests`
4. `itemrequests`
5. `openrequestresponses`
6. `conversations`

Key relations
1. `Listing.ownerId -> User`
2. `RentalRequest.listingId/renterId/lenderId -> Listing/User`
3. `ItemRequest.requesterId/lenderId/listingId -> User/Listing`
4. `Conversation.requestId OR itemRequestId + renterId + lenderId`

Key indexes
1. Listing: `category`, `city`, `locality`, `ownerId`, `availabilityStatus`, `moderationStatus`, `rentPrice`.
2. RentalRequest: `(renterId, status, createdAt)`, `(lenderId, status, createdAt)`.
3. ItemRequest: `(requesterId, createdAt)`, `(category, city, status)`, `expiresAt`.
4. Conversation: `(renterId, lenderId, updatedAt)`, unique sparse `requestId`.

Frontend Events
1. Session hydrate/login/logout.
2. City selection and geolocation detection.
3. Search submit, suggestion click, filter updates.
4. Listing creation with angle-based photo upload.
5. Request creation from listing modal.
6. Open request creation from request wizard.
7. Lender response to open request.
8. Chat message send and quick-action message.
9. Dashboard status action clicks.

Backend Events
1. Auth: `/api/auth/demo-users`, `/api/auth/mock-login`.
2. Listings: `/api/listings`, `/api/listings/facets`, `/api/listings/:id`.
3. Requests: `/api/requests`, `/api/requests/:id/status`.
4. Item Requests: `/api/item-requests/*`, `/api/lender/open-requests`.
5. Conversations: `/api/conversations/*`.
6. Profile/KYC: `/api/users/me`, `/api/users/me/password`, `/api/kyc/mock/*`.
7. Upload: `/api/uploads/images`.

Key Challenges and Mitigation
1. Search relevance misses for user wording/synonyms.
   1. Mitigation: text index + synonym map + weighted relevance for title/subcategory/spec fields.
2. Trust variability across users/listings.
   1. Mitigation: stronger listing quality gates, moderation queue, trust score inputs.
3. Drop-off after request creation.
   1. Mitigation: nudges, lifecycle reminders, SLA indicators.
4. No payment protection in V1.
   1. Mitigation: explicit UX warnings and phased payment roadmap.
5. Admin operational visibility not exposed in UI.
   1. Mitigation: enable admin frontend screens for existing APIs.

Constraints
1. Mock auth and mock KYC in current version.
2. Polling refresh for chat (no real-time socket layer).
3. Upload storage is local filesystem URL, no CDN/signed URL strategy.
4. No integrated dispute, refund, payout, or reconciliation operations.

Open Questions
1. Should open requests accept multiple competing offers until renter finalizes one?
2. What mandatory trust gates should apply by item value threshold?
3. Should price comparison normalize all pricing options to a common baseline?
4. What is the target dispute handling SLA and ownership model?
5. Should listing moderation default to `pending` (instead of current approved-on-create path)?

Design Link
Current: no Figma link found in repo.
Recommendation: establish one canonical design source with interaction/motion to

kens.

App Onboarding
Objective:
1. Introduce product and capture key permissions/state before discovery.

Current status:
1. No dedicated onboarding flow.
2. Location access is requested from navbar geolocation control and optional aut
o-detect behavior.

Suggested:
1. Onboarding carousel (3 cards).
2. Explicit location prompt with fallback.
3. Optional notification prompt and persisted flags.

Homepage
Objective:
1. Trust-led discovery and quick navigation to category/search paths.

Implemented:
1. Hero with animated cards, search input, and quick category pills.
2. Category showcase with image-led cards.
3. City-aware "Fresh Listings Near You" feed.

Gaps:
1. Dynamic marketing CMS content not wired.
2. Wallet/wishlist modules not present.

Mega Menu
Objective:
1. Structured and hierarchical browse.

Current equivalent:
1. Explore page + category pages + filters.

Gap:
1. No dedicated mega-menu panel/toggle experience.

Marketing Module
Objective:
1. Manage dynamic homepage and educational content surfaces.

Current:
1. Static/embedded content.

Recommended:
1. CMS-driven banners, trust blocks, onboarding cards, curated collections.

Bottom Navigation
Objective:
1. Fast mobile-first access to key journeys.

Current:
1. Sticky top navbar and floating quick-action FAB.

Recommended:
1. Mobile bottom nav tabs for Home/Search/Requests/Chat/Profile.

Search
Objective:
1. Fast and high-relevance listing discovery.

Implemented:
1. Query-driven search with left filters + right results.
2. Facet APIs and dynamic filter chips.
3. Category-specific and city/locality filters.
4. Empty-state handling for no results.

Flow:
`User query/filter -> route params -> listings + facets API -> render results ->
 listing detail or request path`

AI Chatbot
Objective:
1. Conversational discovery and guidance.

Current:
1. Not implemented (only user-to-user chat is present).

Reel Section
Objective:
1. Discovery through short-form content.

Current:
1. Not implemented.

Service Education Page (RENTeasy equivalent: category/item guidance)
Objective:
1. Educate users on safe handling, suitability, and checklist-based rental decis
ions.

Current:
1. No dedicated education module page.
2. Safety messaging appears at listing/chat levels.

Profile Section
Objective:
1. Centralize profile quality and account management.

Implemented:
1. Profile edit (name, email, phone, photo).
2. Password update.
3. KYC status visibility and mock transition endpoints.

Clinic Description Page (RENTeasy equivalent: Listing Detail Page)
Objective:
1. Convert discovery intent into actionable request/chat.

Implemented:
1. Gallery, pricing, owner/trust badges, condition/replacement/deposit/availabil
ity.
2. Rules, accessories, and safety banner.
3. Request CTA that moves to chat after request creation.

Slot Booking Flow (RENTeasy equivalent: Date-based Rental Request Flow)
Objective:
1. Collect rental dates/purpose and initiate structured agreement lifecycle.

Implemented:
`Request submit -> status=requested -> chat -> accepted -> confirmed -> active -
> return_pending -> completed`

Payment Flow
Objective:
1. Secure and track rent/deposit payments with settlement.

Current:
1. Not implemented.

Recommended:
`Request acceptance -> payment hold/capture -> lifecycle-linked settlement -> re
fund/dispute path`

Customer Service
Objective:
1. Provide support and issue resolution pathways.

Implemented:
1. Help/Privacy/Terms/Contact pages.
2. Trust/safety UI prompts.

Not yet implemented:
1. Ticketing backend and SLA queue.
2. Integrated complaint/dispute workflow.